

Market Insights

Real-Time Market Intelligence Pipeline with NLP and RAG
Technical Foundations and Architecture

Doctum Consilium
Generated 2026-05-08

Abstract

Market data ingestion, NLP summarisation, and Retrieval-Augmented Generation pipeline powered by Apache Airflow, Python, and Docker. This document covers the theoretical foundations, architectural decisions, and key technical concepts required to understand, contribute to, and maintain this repository.

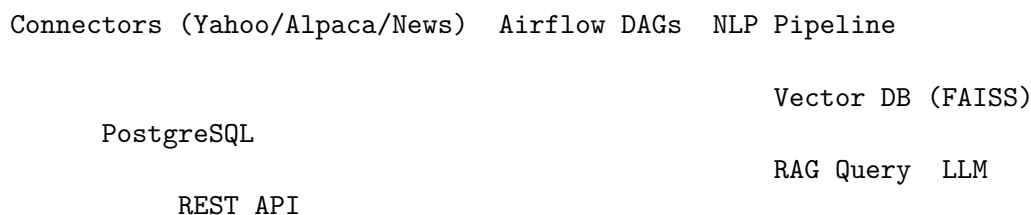
Contents

Overview

Market Insights is a data engineering and NLP pipeline that collects financial news and price data, extracts key insights via extractive summarisation and LLM-augmented RAG, and serves structured signals to downstream consumers.

The pipeline is orchestrated by Apache Airflow and runs on a containerised stack (Docker Compose). Raw data from multiple connectors (Yahoo Finance, Alpaca, economic calendars) is stored in a vector database for semantic search.

Architecture



Key Technical Concepts

- Apache Airflow: directed acyclic graph (DAG) scheduling; task dependencies, XCom for inter-task communication, operator types (PythonOperator, BashOperator, HttpSensor).
- Extractive Summarisation: TF-IDF sentence scoring; token frequency weighted by inverse document frequency; $score(s) = \sum_{t \in s} tf(t) \cdot idf(t)$.
- Vector Embeddings: sentence-transformers for semantic encoding; FAISS approximate nearest-neighbour (ANN) search; cosine similarity $\cos \theta = \frac{u \cdot v}{|u||v|}$.
- RAG (Retrieval-Augmented Generation): relevant context retrieved from vector store is injected into the LLM prompt to ground responses in factual financial data.
- Data Quality: price integrity tests (OHLCV consistency), connector health checks, deduplication by hash.

Data Flow

Airflow DAG (scheduled) → Connector fetch → HTML strip + NLP clean → Embedding generation → FAISS upsert → On-demand RAG query → LLM enrichment → REST response.

Technology Stack

Component	Technology
Orchestration	Apache Airflow 2.x
NLP	sentence-transformers, spaCy
Vector store	FAISS (CPU)
LLM proxy	LiteLLM
Data sources	Yahoo Finance, Alpaca News API
Database	PostgreSQL
Container	Docker Compose

Design Decisions

- Airflow chosen over cron for retries, visibility, and dependency management.
- FAISS (CPU) preferred over managed vector DBs to avoid cloud egress costs.
- Extractive summarisation is language-model-free and deterministic — useful for offline/low-latency paths.

Repository Structure

Listing 1: Top-level directory layout

```
market_insights/  
  .github/  
    copilot-instructions.md    # GitHub Copilot guardrails  
    agents/  
      ecr-secrets-agent.agent.md # ECR secret rotation runbook  
  CLAUDE.md                   # Claude Code guardrails  
  GEMINI.md                   # Gemini CLI guardrails  
  INFRASTRUCTURE.md           # Operational runbook  
  ROADMAP.md                  # Execution log and roadmap  
  README.md                   # Project overview
```

Contributing

1. Read README.md, INFRASTRUCTURE.md, and ROADMAP.md before any task.
2. Follow the Code Documentation Standard: every public function must have a docstring (Google-style for Python, JSDoc for TypeScript, XML for C#).
3. Run the guardrails validator before committing:

```
git add -A && ./guardrails/bin/validate_guardrails.sh
```

4. For deployment or destructive operations, always use a named tmux session:

```
tmux new-session -A -s deploy-market_insights
```